

# Apuntes/Documentación Desarrollo

## Trabajo Semestral – Grupo 9

### Integrantes

|                   |            |
|-------------------|------------|
| Joaquín Adaury    | 2025405466 |
| Joaquín Navarrete | 2025420121 |
| Vicente Vergara   | 2025431734 |

### Enunciado

|                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                     |
|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Tema 2: Simulador de Tienda de Mascotas Virtual (Referente: Angie Ramirez)</b>                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                   |
| Este simulador ofrecerá al usuario la experiencia de gestionar su propia tienda de mascotas virtual. El jugador comenzará con un presupuesto inicial para adquirir diferentes tipos de mascotas (ej. perros, gatos, peces, pájaros), cada una con atributos y necesidades específicas como alimentación (tipo de comida, frecuencia), higiene, nivel de felicidad, tipo de recinto y salud. El usuario deberá gestionar el inventario de mascotas y de suministros (comida, medicinas). Las interacciones incluirán alimentar a las mascotas, limpiar sus hábitats, jugar con ellas para mantener su felicidad, y atender su salud. El estado de cada mascota deberá ser visible y cambiará según los cuidados recibidos. El sistema también simulará la llegada de clientes virtuales interesados en adoptar mascotas, permitiendo al jugador venderlas y así obtener ingresos para continuar gestionando y expandiendo la tienda. |

- La exigencia explícita de incluir "diferentes tipos de mascotas" (ej. perros, gatos, peces) que comparten características generales, pero poseen "necesidades específicas", sugirió inmediatamente la implementación de una jerarquía de clases robusta.

- El núcleo interactivo del simulador radica en la variabilidad de los atributos de cada animal. Conceptos como nivel de felicidad, higiene, salud y alimentación indican que los objetos no serán estáticos. Se infiere el uso de encapsulamiento estricto y métodos de validación para asegurar que las estadísticas no sobrepasen límites lógicos (ej. evitar que la salud caiga por debajo del 0% o supere el 100%).

- Administrar colecciones de datos dinámicas para lo que es el inventario mencionado, El controlador del sistema deberá ser capaz manejar simultáneamente al menos dos inventarios distintos: uno para las entidades vivas (mascotas) y otro para los recursos consumibles (suministros como comida y medicinas).

- La progresión del sistema está anclada a un flujo de caja continuo. El ciclo lógico interpretado consiste en:

*Adquirir Animales → Realizar Cuidados → Adoptar a clientes virtuales*

Siendo el objetivo principal generar rentabilidad y permitir la expansión comercial como lo puede ser mejorando espacios visuales o estadísticas como lo pueden ser inventarios y sus límites.

- El flujo del programa no puede permitir acciones no reguladas. Operaciones como la venta de una mascota o la compra de insumos deben estar sujetas a validaciones de estado (ej. Verificar Economía o nivel de salud del animal antes de autorizar la adopción).

Ideas para implementar aparte del código principal

Sistema de Persistencia (Guardado/Carga de Partida): Buscaremos implementar serialización de archivos (.dat o texto) para que el jugador pueda guardar el progreso de su tienda (presupuesto e inventario actual) y retomarlo en futuras sesiones, evitando que la simulación se reinicie en cada ejecución.

Progresión y Expansión Comercial (Upgrades): En lugar de tener un límite estático, se propuso que el jugador pueda reinvertir sus ganancias para "comprar expansiones" del local. Esto modificaría dinámicamente el límite de capacidad máxima (`inventariomaximo`), premiando la buena gestión financiera.

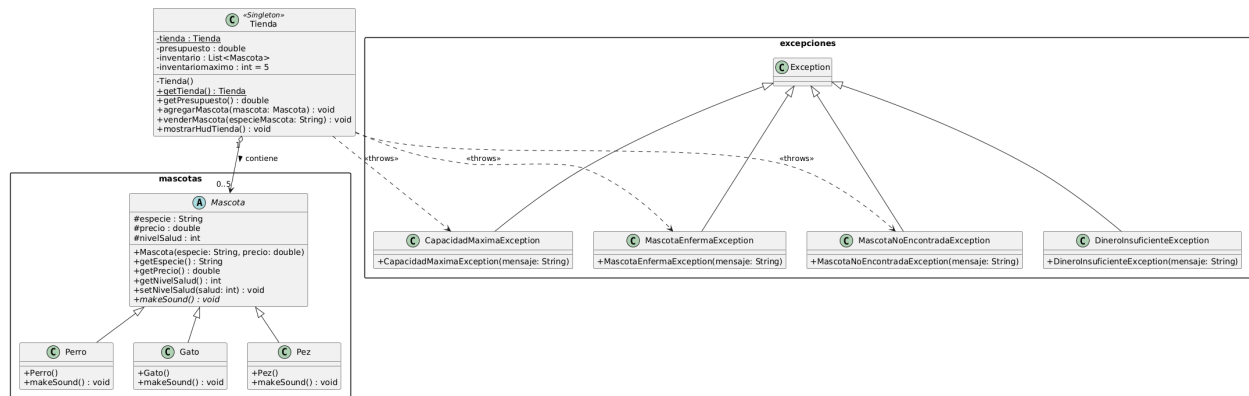
Eventos Aleatorios y Clientes VIP o exigentes: Para hacer la venta más interactiva, se conceptualizó un sistema de "llegada de clientes" que ofrezcan montos variables por mascota, o clientes especiales que busquen una especie con un nivel de salud o felicidad específico, obligando al jugador a utilizar suministros estratégicamente antes de vender.

Ciclo de Tiempos o Ticks: Un motor de tiempo interno donde cada acción realizada en la tienda (o cada cierto tiempo en la interfaz gráfica) reduzca gradualmente las estadísticas de los animales (aumentando el hambre y reduciendo la higiene), simulando necesidades biológicas reales.

## Semana 1

Durante esta semana, se optimizó el diagrama de casos de uso y planeamos nuestro primer diagrama UML, junto a la elección de patrones de diseños e integración u proyección de estos mismos:

Diagrama UML Semana 1 (LINKEADO DENTRO DE REPOSITORIO DE GITHUB)



## Modelado del Código Inicial

Para representar el catálogo de la tienda de una manera escalable, se implementó una jerarquía estricta

Clase Abstracta Mascota: Actúa como principal dentro del dominio, se definen atributos protegidos (protected) como especie, precio y nivelSalud para permitir el acceso a las clases hijas, manteniendo el encapsulamiento.

Se implementan Setters defensivos para anticipar la lógica del cuidado, agregando métodos de mutación que incorporan validaciones matemáticas para garantizar la integridad referencial, impidiendo que estadísticas como las alud excedan el 100% o caigan por debajo del 0%

Se definió el método abstracto makeSound(), obligando a que cada entidad concreta (Perro, Gato, Pez) sobrescriba (realice @Override) su propio comportamiento.

Para asegurar la continuidad del programa y evitar interrupciones abruptas, se construyó un paquete aislado de Excepciones personalizadas para mantener detalles como:

La Integridad del Inventario (CapacidadMaximaException): Restringe el desbordamiento de las listas evaluando el tamaño de la colección contra una constante máxima, abortando la inserción si el límite ha sido alcanzado.

Seguridad Transaccional (MascotaNoEncontradaException, MascotaEnfermaException): Actúan como *guardrails* durante la venta. Si el animal buscado no existe o su salud se encuentra en un umbral crítico (menor al 50%), la transacción se anula preventivamente.

## Patrones de Diseños Elegidos

- Patrón Singleton (Implementado)

Se utiliza Singleton para evitar la multiplicidad de estados globales. En nuestro contexto para la simulación económica y de gestión de recursos, instanciar por error múltiples

"Tiendas" provocaría la desincronización del dinero y duplicación o pérdida de mascotas. La utilizamos para garantizar que, durante toda la ejecución del programa, tanto la interfaz gráfica en SWING como los bloques de prueba interactúen con una única caja registradora (presupuesto de \$1000 inicial) y un único inventario de listas.

La clase Tienda posee un constructor privado, almacena su propia instancia estática (private static Tienda tienda) y provee un punto de acceso global mediante el método getTienda().

#### - Patrón Factory Method

Para cumplir con el principio *Open/Closed* (Abierto a extensión, cerrado a modificación) y desacopla la lógica de la interfaz gráfica (SWING) de la creación de los objetos concretos. En lugar de que los botones de la ventana ejecuten comandos rígidos como new Perro() o new Pez(), la interfaz simplemente envía un String con el nombre de la especie deseada. La clase se encarga de instanciar el objeto correcto con sus atributos base, facilitando la futura expansión del catálogo de la tienda sin tocar el código de la ventana principal.

## Implementación del Código Fuente Base

A continuación, se detalla la primera iteración del código:

### *Package Excepciones*

```
package excepciones;

public class CapacidadMaximaException extends Exception {
    public CapacidadMaximaException(String mensaje) {
        super(mensaje);
    }
}

public class MascotaEnfermaException extends Exception {
    public MascotaEnfermaException(String mensaje) {
        super(mensaje);
    }
}

public class MascotaNoEncontradaException extends Exception {
    public MascotaNoEncontradaException(String mensaje) {
        super(mensaje);
    }
}
```

### *Package Mascotas*

Contiene la jerarquía de las entidades y el patrón creacional para su instanciación:

#### Clase base

```
package mascotas;

public abstract class Mascota {
    protected String especie;
    protected double precio;
    protected int nivelSalud;

    public Mascota(String especie, double precio) {
        this.especie = especie;
        this.precio = precio;
        this.nivelSalud = 100;
    }

    public String getEspecie() { return especie; }
    public double getPrecio() { return precio; }
    public int getNivelSalud() { return nivelSalud; }

    public void setNivelSalud(int salud) {
        this.nivelSalud = Math.max(0, Math.min(100, salud));
    }

    public abstract void makeSound();
}
```

#### Ejemplo de Clase Concreta:

```
package mascotas;

public class Perro extends Mascota {
    public Perro() { super("Perro", 150.0); }

    @Override
    public void makeSound() { System.out.println("¡Woof Woof! El perro se mueve emocionado."); }
}
```

### *Controlador Principal Tienda.java*

Motor central del simulador que administra el flujo económico y las colecciones aplicando el patrón Singleton:

```
import java.util.ArrayList;
import java.util.List;
import excepciones.*;
import mascotas.Mascota;

public class Tienda {
    private static Tienda tienda;
    private double presupuesto;
    private List<Mascota> inventario;
    private final int inventariomaximo = 5;

    private Tienda() {
        this.presupuesto = 1000.0;
        this.inventario = new ArrayList<>();
    }

    public static Tienda getTienda() {
        if (tienda == null) {
            tienda = new Tienda();
        }
        return tienda;
    }

    public double getPresupuesto() { return presupuesto; }

    public void agregarMascota(Mascota mascota) throws CapacidadMaximaException {
        if (inventario.size() >= inventariomaximo) {
            throw new CapacidadMaximaException("No hay espacio en la tienda para un/a " +
mascota.getEspecie());
        }
        inventario.add(mascota);
    }

    public void venderMascota(String especieMascota) throws
MascotaNoEncontradaException, MascotaEnfermaException {
        Mascota mascotaVender = null;

        for (Mascota m : inventario) {
            if (m.getEspecie().equalsIgnoreCase(especieMascota)) {
                mascotaVender = m;
            }
        }
    }
}
```

```

        break;
    }
}

if (mascotaVender == null) {
    throw new MascotaNoEncontradaException("No tienes ningún " + especieMascota
+ " en tu inventario para la venta.");
}
if (mascotaVender.getNivelSalud() < 50) {
    throw new MascotaEnfermaException("El " + especieMascota + " está demasiado
débil para adoptarlo.");
}

inventario.remove(mascotaVender);
double precioVenta = mascotaVender.getPrecio() * 1.5;
this.presupuesto += precioVenta;
}
}

```

## Test Unitarios Core

Durante esta etapa inicial, el enfoque de las pruebas mediante un Test Harness, se verificó de forma aislada que los métodos de mutación de la clase Mascota funcionen correctamente, impidiendo mediante validaciones matemáticas que los atributos de salud o hambre superen el 100% o caigan a números negativos.

Asimismo, se forzó el desbordamiento del inventario en el controlador principal para comprobar que el sistema rechace inserciones no válidas levantando la excepción CapacidadMaximaException, protegiendo así la memoria y confirmando que el patrón Singleton mantiene una única instancia aislada.

## Código Test Unitario

```

import excepciones.*;
import mascotas.*;
import org.junit.jupiter.api.Test;
import static org.junit.jupiter.api.Assertions.*;

public class CoreSemana1Test {

    /**
     * T1: Prueba de Encapsulamiento y Límites Lógicos.
     * Verifica que los setters defensivos de la clase abstracta Mascota
     * restrinjan correctamente los valores anómalos usando Math.max y Math.min.
     */
}

```

```

@Test
public void testLimitesAtributosMascota() {
    Mascota perroTest = new Perro(); // La salud inicial debe ser 100

    // desbordamiento superior
    perroTest.setNivelSalud(150);
    assertEquals(100, perroTest.getNivelSalud(), "Fallo: La salud logró superar el límite
máximo del 100%.");

    // Intento de desbordamiento inferior (números negativos)
    perroTest.setNivelSalud(-20);
    assertEquals(0, perroTest.getNivelSalud(), "Fallo: La salud logró caer por debajo del
límite mínimo de 0%.");
}

/**
 * T2: Prueba del Patrón Singleton.
 * Valida que múltiples llamadas al método estático getTienda()
 * retornen exactamente la misma referencia en el espacio de memoria (Heap).
 */
@Test
public void testInstanciaUnicaSingleton() {
    Tienda instanciaA = Tienda.getTienda();
    Tienda instanciaB = Tienda.getTienda();

    assertEquals(instanciaA, instanciaB, "Fallo Crítico: El patrón Singleton está creando
múltiples instancias de la Tienda.");
}

/**
 * T3: Prueba de Tolerancia a Fallos (Límite de Inventario).
 * Fuerza un escenario de sobrecapacidad para asegurar que el sistema
 * levante la CapacidadMaximaException en lugar de corromper la lista.
 */
@Test
public void testExcepcionDesbordamientoInventario() {
    Tienda tiendaTest = Tienda.getTienda();

    // AssertThrows verifica que el bloque de código interno dispare la excepción
esperada
    assertEquals(CapacidadMaximaException.class, () -> {
        // Se asume un límite de 5. Se intenta ingresar un número mayor para forzar el
error.
        for (int i = 0; i < 10; i++) {

```

```

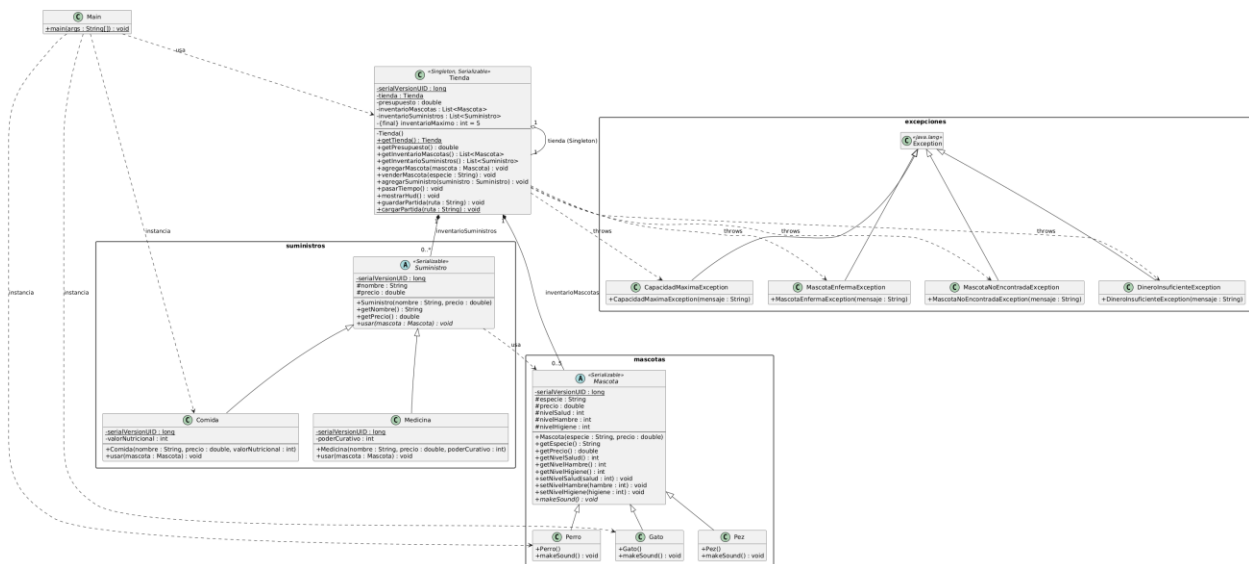
        tiendaTest.agregarMascota(new Gato());
    }
    }, "Fallo: El sistema permitió ingresar mascotas por sobre el límite máximo de la
tienda.");
}
}

```

## Semana 2

Durante esta semana, el enfoque principal será la expansión del catálogo de la tienda mediante la inclusión de ítems consumibles y el desarrollo de un sistema de persistencia de datos (File I/O) para permitir el guardado y recuperación de la partida del jugador, también la implementación de una degradación en base al tiempo.

Diagrama UML Semana 2 (LINKEADO DENTRO DE REPOSITORIO DE GITHUB)



## Expansión del Modelo (Paquete Suministros)

Se diseñó un nuevo paquete destinado a los recursos que el jugador puede comprar para el cuidado del inventario de Mascotas aplicando los mismos principios de abstracción de la semana anterior

La clase base abstracta Suministro, definiendo atributos comunes como nombre y precio, y un método abstracto usar(Mascota mascota), se implementaron las clases concretas Comida (que regenera positivamente el atributo de hambre) y Medicina (que restaura el

atributo de salud). Esto permite que los objetos interactúen entre sí sin romper el encapsulamiento, delegando la modificación de estadísticas a métodos controlados.

## Sistema de Persistencia (Guardado y Carga de Partida)

Para entregarle al simulador una experiencia de uso continua, se integró la capacidad de almacenar el estado dinámico del juego (Dinero actual y animales del inventario) en el disco local del jugador.

Serialización de Objetos: Se modificaron las clases principales de la jerarquía (Tienda, Mascota, Suministro) mediante la interfaz nativa `java.io.Serializable`. Esto permite convertir el estado de los objetos en un flujo de bytes (*byte stream*).

Gestión de Archivos (I/O Streams): Se programaron métodos transaccionales utilizando `ObjectOutputStream` para la escritura (guardado) y `ObjectInputStream` para la lectura (carga) de archivos `.dat`.

Manejo de Excepciones de Entrada/Salida: Toda la lógica de persistencia fue envuelta en bloques `try-catch` para capturar `IOException` y `ClassNotFoundException`, evitando que el simulador colapse si el archivo de guardado se corrompe, es eliminado o el sistema carece de permisos de escritura.

## Implementación de Pruebas Unitarias

Se validó el correcto funcionamiento de la persistencia de datos, comprobando que la deserialización de archivos `.dat` restaure la memoria del inventario y el presupuesto exactamente igual tras reiniciar el programa. También se testeó el motor de tiempo, asegurando que el paso del tiempo altere correctamente las estadísticas vitales de los animales. Finalmente, se simularon compras sin saldo suficiente para confirmar que la `DineroInsuficienteException` actúe como barrera transaccional, bloqueando cualquier operación que deje el balance de la tienda en números negativos.

### Código Test Unitarios

```
import excepciones.*;
import mascotas.Gato;
import mascotas.Perro;
import suministros.Comida;
import suministros.Medicina;
import suministros.Suministro;

import org.junit.jupiter.api.*;
import static org.junit.jupiter.api.Assertions.*;
```

```
import java.io.File;
import java.lang.reflect.Field;

class TiendaTest {

    private static final String ARCHIVO_TEST = "test_partida.dat";
    // Resetea Singleton por reflexión antes de cada test para aislar el estado.
    @BeforeEach
    void resetSingleton() throws Exception {
        new File(ARCHIVO_TEST).delete();
        Field campo = Tienda.class.getDeclaredField("tienda");
        campo.setAccessible(true);
        campo.set(null, null);
    }

    @AfterAll
    static void limpiarArchivos() {
        new File(ARCHIVO_TEST).delete();
    }

    @Test
    @DisplayName("guardarPartida + cargarPartida restauran presupuesto e inventario")
    void persistencia_restauraEstadoCompleto() throws Exception {
        Tienda original = Tienda.getTienda();
        original.agregarMascota(new Perro());
        original.agregarMascota(new Gato());
        original.agregarSuministro(new Comida("Croquetas", 100.0, 30));

        double presupuestoEsperado = original.getPresupuesto();
        int mascotasEsperadas = original.getInventarioMascotas().size();
        int suministrosEsperados = original.getInventarioSuministros().size();

        original.guardarPartida(ARCHIVO_TEST);

        // Simulamos reiniciar el programa para dejar Singleton en null.
        Field campo = Tienda.class.getDeclaredField("tienda");
        campo.setAccessible(true);
        campo.set(null, null);

        Tienda.cargarPartida(ARCHIVO_TEST);
        Tienda cargada = Tienda.getTienda();

        assertEquals(presupuestoEsperado, cargada.getPresupuesto(), 0.001,
```

```
        "El presupuesto debe sobrevivir al reinicio");
        assertEquals(mascotasEsperadas, cargada.getInventarioMascotas().size(),
            "El inventario de mascotas debe recuperarse");
        assertEquals(suministrosEsperados, cargada.getInventarioSuministros().size(),
            "El inventario de suministros debe recuperarse");
    }
}
```

```
@Test
@DisplayName("cargarPartida con archivo inexistente inicia una tienda nueva")
void persistencia_archivoInexistente_creaTiendaNueva() {
    Tienda.cargarPartida("archivo_que_no_existe_" + System.nanoTime() + ".dat");
    Tienda t = Tienda.getTienda();

    assertNotNull(t);
    assertEquals(1000.0, t.getPresupuesto(), 0.001);
    assertTrue(t.getInventarioMascotas().isEmpty());
}
}
```

// tiempo, pasarTiempo altera las estadística

```
@Test
@DisplayName("pasarTiempo aumenta el hambre y reduce la higiene")
void tiempo_degradaHambreEHigiene() throws Exception {
    Tienda t = Tienda.getTienda();
    Perro p = new Perro();
    t.agregarMascota(p);

    int hambreInicial = p.getNivelHambre();
    int higienInicial = p.getNivelHigiene();

    t.pasarTiempo();
    assertTrue(p.getNivelHambre() > hambreInicial,
        "El hambre debe subir tras pasar el tiempo");
    assertTrue(p.getNivelHigiene() < higienInicial,
        "La higiene debe caer tras pasar el tiempo");
}
}
```

```
@Test
@DisplayName("Mascota con hambre extrema pierde salud al pasar el tiempo")
void tiempo_hambreExtremaCastigaSalud() throws Exception {
    Tienda t = Tienda.getTienda();
    Perro p = new Perro();
    p.setNivelHambre(85); // Ya supera el umbral (>80) tras el tick.
}
```

```

        t.agregarMascota(p);

        int saludInicial = p.getNivelSalud();
        t.pasarTiempo();

        assertTrue(p.getNivelSalud() < saludInicial,
            "La salud debe caer cuando el hambre está muy alta");
    }

    @Test
    @DisplayName("Los niveles de estadísticas nunca superan sus límites (0-100)")
    void tiempo_estadisticasSeMantienenEnRango() throws Exception {
        Tienda t = Tienda.getTienda();
        Perro p = new Perro();
        t.agregarMascota(p);

        for (int i = 0; i < 20; i++) t.pasarTiempo();

        assertTrue(p.getNivelHambre() >= 0 && p.getNivelHambre() <= 100);
        assertTrue(p.getNivelHigiene() >= 0 && p.getNivelHigiene() <= 100);
        assertTrue(p.getNivelSalud() >= 0 && p.getNivelSalud() <= 100);
    }

    // DinerolInsuficienteException
    @Test
    @DisplayName("Comprar por encima del presupuesto lanza DinerolInsuficienteException")
    void dinero_compraSinSaldo_lanzaExcepcion() {
        Tienda t = Tienda.getTienda();
        Suministro carisimo = new Comida("Alimento de Oro", 999_999.0, 100);

        assertThrows(DinerolInsuficienteException.class,
            () -> t.agregarSuministro(carisimo));
    }

    @Test
    @DisplayName("Compra fallida no modifica presupuesto ni inventario (rollback lógico)")
    void dinero_compraSinSaldo_noAlteraEstado() {
        Tienda t = Tienda.getTienda();
        double presupuestoInicial = t.getPresupuesto();
        int suministrosPrevios = t.getInventarioSuministros().size();

        Suministro carisimo = new Comida("Alimento de Oro", 999_999.0, 100);

```

```

    assertThrows(DineroInsuficienteException.class,
        () -> t.agregarSuministro(carisimo));

    assertEquals(presupuestoInicial, t.getPresupuesto(), 0.001,
        "El presupuesto NO debe cambiar si la compra falla");
    assertEquals(suministrosPrevios, t.getInventarioSuministros().size(),
        "El inventario NO debe crecer si la compra falla");
    assertTrue(t.getPresupuesto() >= 0,
        "El presupuesto nunca debe quedar en negativo");
}

@Test
@DisplayName("Compra dentro del presupuesto descuenta el precio correcto")
void dinero_compraValida_descuentaCorrecto() throws DineroInsuficienteException {
    Tienda t = Tienda.getTienda();
    double presupuestoInicial = t.getPresupuesto();
    Suministro comida = new Comida("Alimento estándar", 50.0, 20);

    t.agregarSuministro(comida);

    assertEquals(presupuestoInicial - 50.0, t.getPresupuesto(), 0.001);
    assertEquals(1, t.getInventarioSuministros().size());
}

// Capacidad y uso de suministros
@Test
@DisplayName("Superar la capacidad máxima lanza CapacidadMaximaException")
void capacidad_llena_lanzaExcepcion() throws CapacidadMaximaException {
    Tienda t = Tienda.getTienda();
    for (int i = 0; i < 5; i++) {
        t.agregarMascota(new Perro());
    }
    assertThrows(CapacidadMaximaException.class,
        () -> t.agregarMascota(new Gato()));
}

@Test
@DisplayName("Comida.usar reduce el nivel de hambre por su valor nutricional")
void comida_usar_reduceHambre() {
    Perro p = new Perro();
    p.setNivelHambre(50);
    Comida snack = new Comida("Snack", 10.0, 30);

```

```

        snack.usar(p);

        assertEquals(20, p.getNivelHambre());
    }

    @Test
    @DisplayName("Medicina.usar aumenta el nivel de salud por su poder curativo")
    void medicina_usar_aumentaSalud() {
        Gato g = new Gato();
        g.setNivelSalud(50);
        Medicina antibiotico = new Medicina("Antibiótico", 40.0, 30);

        antibiotico.usar(g);

        assertEquals(80, g.getNivelSalud());
    }
}

```

### Resultados Test Unitarios

|                                                                          |       |
|--------------------------------------------------------------------------|-------|
| ✓ TiendaTest                                                             | 46 ms |
| ✓ Superar la capacidad máxima lanza CapacidadMaximaException             | 15 ms |
| ✓ Comida.usar reduce el nivel de hambre por su valor nutricional         | 2 ms  |
| ✓ Mascota con hambre extrema pierde salud al pasar el tiempo             |       |
| ✓ Compra fallida no modifica presupuesto ni inventario (rollback lógico) | 2 ms  |
| ✓ Comprar por encima del presupuesto lanza DinerolInsuficienteException  |       |
| ✓ guardarPartida + cargarPartida restauran presupuesto e inventario      | 26 ms |
| ✓ cargarPartida con archivo inexistente inicia una tienda nueva          | 1 ms  |
| ✓ Los niveles de estadísticas nunca superan sus límites (0-100)          |       |
| ✓ Compra dentro del presupuesto descuenta el precio correcto             |       |
| ✓ pasarTiempo aumenta el hambre y reduce la higiene                      |       |
| ✓ Medicina.usar aumenta el nivel de salud por su poder curativo          |       |

### Lógica de Degradación y Ciclo de Vida

Como mencionamos anteriormente, buscamos darle dinamismo a la simulación y alejarse de una base de datos estática, implementamos el Sistema de Ticks proyectado en la etapa inicial. Se desarrolló un motor para la degradación que itera sobre la colección de mascotas, reduciendo de manera gradual sus niveles de higiene y aumentando su hambre. Esto obliga al jugador a interactuar constantemente con el sistema y reinvertir parte de su presupuesto en ítems del package Suministros (Comida y Medicina), complementando al ciclo económico del juego.

## Implementación del Código

A continuación, se detallan las implementaciones clave:

### *Gestión de Archivos en Tienda.java*

Se integraron los métodos de guardado y carga directamente en el Singleton de la tienda, permitiendo volcar todo el estado global a un archivo.

```
import java.io.*;
// ... (resto de importaciones)

public class Tienda implements Serializable {
    // Es crucial definir un serialVersionUID para evitar errores de versión al cargar la
    partida
    private static final long serialVersionUID = 1L;

    // ... (atributos y métodos Singleton existentes) ...

    // Método para Guardar la Partida
    public void guardarPartida(String rutaArchivo) {
        try (ObjectOutputStream oos = new ObjectOutputStream(new
        FileOutputStream(rutaArchivo))) {
            oos.writeObject(this);
            System.out.println("Partida guardada exitosamente en: " + rutaArchivo);
        } catch (IOException e) {
            System.err.println("Error crítico al guardar la partida: " + e.getMessage());
        }
    }

    // Método estático para Cargar la Partida
    public static Tienda cargarPartida(String rutaArchivo) {
        try (ObjectInputStream ois = new ObjectInputStream(new
        FileInputStream(rutaArchivo))) {
            Tienda tiendaCargada = (Tienda) ois.readObject();
            tienda = tiendaCargada; // Actualizamos la instancia Singleton
            System.out.println("Partida cargada exitosamente desde: " + rutaArchivo);
            return tienda;
        } catch (FileNotFoundException e) {
            System.err.println("No se encontró ningún archivo de guardado previo.");
        } catch (IOException | ClassNotFoundException e) {
            System.err.println("Error al cargar la partida: " + e.getMessage());
        }
        return getTienda(); // Si falla, retorna una tienda nueva/vacía
    }
}
```

```
}
```

### *Adaptación de Entidades Mascota.java*

La lista de inventario pudiera ser guardada por la Tienda, fue obligatorio que toda la jerarquía de animales fuera serializable.

```
package Mascotas;
import java.io.Serializable;

// Se implementa Serializable en la base para que Perro, Gato y Pez lo hereden
// automáticamente
public abstract class Mascota implements Serializable {
    private static final long serialVersionUID = 1L;

    protected String especie;
    protected double precio;
    protected int nivelSalud;
    // ... (resto del código de Mascota)
}
```

## Semana 3

El objetivo principal de esta semana fue transicionar el simulador desde un entorno de consola basado en texto hacia una aplicación de escritorio interactiva. Para lograr esto sin romper la lógica central previamente validada, se priorizó la separación de responsabilidades (desacoplamiento) mediante la integración de la biblioteca nativa Java SWING. Este proceso exigió reforzar primero el backend con patrones de diseño GoF que garantizaran un modelo robusto antes de exponerlo en una vista gráfica, siguiendo una arquitectura inspirada en MVC (Modelo-Vista-Controlador).

Diagrama UML Semana 3 (LINKEADO DENTRO DE REPOSITORIO DE GITHUB)



La transición está integrada en el setter defensivo `setNivelSalud()` de `Mascota`, garantizando consistencia sin intervención externa: si `nivelSalud < 50` → `EstadoEnfermo`; si `nivelSalud ≥ 50` → `EstadoSano` (curación mediante `Medicina`).

Adicionalmente, se expone `setEstado(EstadoMascota)` para permitir inyección directa desde la fábrica.

Impacto sobre `Tienda`: El `if` hardcodeado fue eliminado y reemplazado por una única línea de delegación polimórfica: `mascotaVender.getEstado().verificarVenta(mascotaVender)`.

Agregar un nuevo estado futuro no requiere tocar `Tienda` nuevamente.

### *Patrón Factory Method (GoF) Creacional: Fábrica de Mascotas*

Hasta esta semana, las mascotas se instanciaban directamente con `new Perro()` en el cliente (`Main.java`). Este acoplamiento directo rompe el principio de encapsulamiento: el código cliente conoce los detalles de construcción y no existe un punto centralizado para aplicar reglas de negocio durante la creación.

Se eligió el patrón `Factory Method (GoF)` para encapsular dos lógicas económicas de adquisición diferenciadas, sin que el cliente necesite conocer los detalles internos de construcción.

Se creó la clase `FabricaMascotas` con dos métodos de fabricación:

|                                                 |                                                                                                                                                                                           |
|-------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <code>crearMascotaComprada(tipo, tienda)</code> | Descuenta el precio del dinero actual. La mascota nace en <code>EstadoSano</code> (estado por defecto). Lanza <code>DineroInsuficienteException</code> si el presupuesto es insuficiente. |
| <code>crearMascotaRescatada(tipo)</code>        | Gratuita. La mascota nace con <code>EstadoEnfermo</code> inyectado via <code>setEstado()</code> , obligando al jugador a usar <code>Medicina</code> antes de poder venderla.              |

Ambos métodos devuelven el tipo abstracto `Mascota`, ocultando la clase concreta al cliente y permitiendo extensión futura sin modificar el código existente.

### *Archivos Involucrados*

|                                   |                                                                                        |
|-----------------------------------|----------------------------------------------------------------------------------------|
| <code>FabricaMascotas.java</code> | Factory con <code>crearMascotaComprada()</code> y <code>crearMascotaRescatada()</code> |
| <code>Tienda.java</code>          | <code>comprarMascota()</code> y <code>rescatarMascota()</code> usan la fábrica.        |
| <code>Main.java</code>            | Demo completa del ciclo rescate, rehabilitación y venta.                               |

### *Patrón Observer (Estructural de Comportamiento): Sincronización Vista-Modelo*

Sin un mecanismo de notificación, la Vista debería consultar activamente al modelo en cada acción (polling), generando acoplamiento fuerte entre capas.

El patrón Observer (GoF) resuelve esto invirtiendo el flujo de este, el modelo notifica a sus suscriptores ante cualquier cambio de estado, sin conocer los detalles de quién observa ni cuántos observadores existen.

|                  |                     |                                                                                                                                                          |
|------------------|---------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------|
| ObservadorTienda | Observer (interfaz) | Define el contrato actualizar().                                                                                                                         |
| VentanaTienda    | ConcreteObserver    | Implementa actualizar() para refrescar el JTextArea del HUD con el estado actual del modelo.                                                             |
| Tienda           | Subject (Publisher) | Mantiene una lista de observadores. Llama a notificarObservadores() al finalizar cada transacción (venderMascota, pasarTiempo, agregarSuministro, etc.). |

Con este esquema, cualquier acción que modifique el estado de la tienda se refleja automáticamente en la interfaz gráfica, sin que Tienda conozca la existencia de SWING ni de VentanaTienda.

### *Clases Involucradas*

|                       |                   |                                                                                      |
|-----------------------|-------------------|--------------------------------------------------------------------------------------|
| ObservadorTienda.java | Interfaz          | Interfaz Observer con contrato actualizar()                                          |
| Tienda.java           | Subject/Publisher | Lista transient + agregarObservador() + notificarObservadores() llamado en 5 métodos |
| VentanaTienda.java    | ConcreteObserver  | implements ObservadorTienda, actualizar() → actualizarHUD()                          |

### *Interfaz Gráfica SWING*

Para no contaminar la lógica de negocio con código de presentación, se creó la clase VentanaTienda como una capa de Vista independiente. Esta clase no contiene ninguna lógica de negocio: solo recibe datos del modelo y los renderiza. La comunicación de acciones del usuario hacia el modelo se hace a través del Singleton Tienda, respetando el flujo unidireccional de datos.

VentanaTienda ahora extiende JFrame y se organiza en dos áreas funcionales:

|                            |                                                                                                                                                                                 |
|----------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Panel de control (botones) | Acciones disponibles para el jugador: comprar mascota, rescatar mascota, vender, usar suministro, pasar tiempo, guardar/cargar partida. Cada botón dispara un método de Tienda. |
| HUD central (JTextArea)    | Área de solo lectura que muestra el estado completo de la tienda. Se actualiza automáticamente mediante el patrón Observer (Paso 4), sin polling.                               |

Los tres patrones aplicados esta semana actúan de forma complementaria. State y Factory Method fortalecen el backend garantizando comportamiento correcto por construcción. Observer desacopla el modelo de la vista.

Mientras que SWING provee la capa de presentación sin contaminar la lógica central. En conjunto, la arquitectura resultante respeta con una alta cohesión y el principio de responsabilidad única (SRP) en cada capa.

El método actualizarHUD() en VentanaTienda está intencionalmente expuesto como public, será el punto exacto donde el Patrón Observer enganchará la notificación de Tienda.

|                             |                                                                                                                                                                                                            |
|-----------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Header                      | Título de la aplicación                                                                                                                                                                                    |
| HUD Central (Terminal)      | Estado completo: presupuesto, mascotas con barras  de salud/hambre/higiene, estado [✓ SANO] / [X ENFERMO], suministros |
| Panel de botones (Derecha)  | 5 secciones: Adquisición · Ventas · Suministros · Tiempo · Sistema                                                                                                                                         |
| Registro de Eventos (Abajo) | Log cronológico de todas las acciones                                                                                                                                                                      |

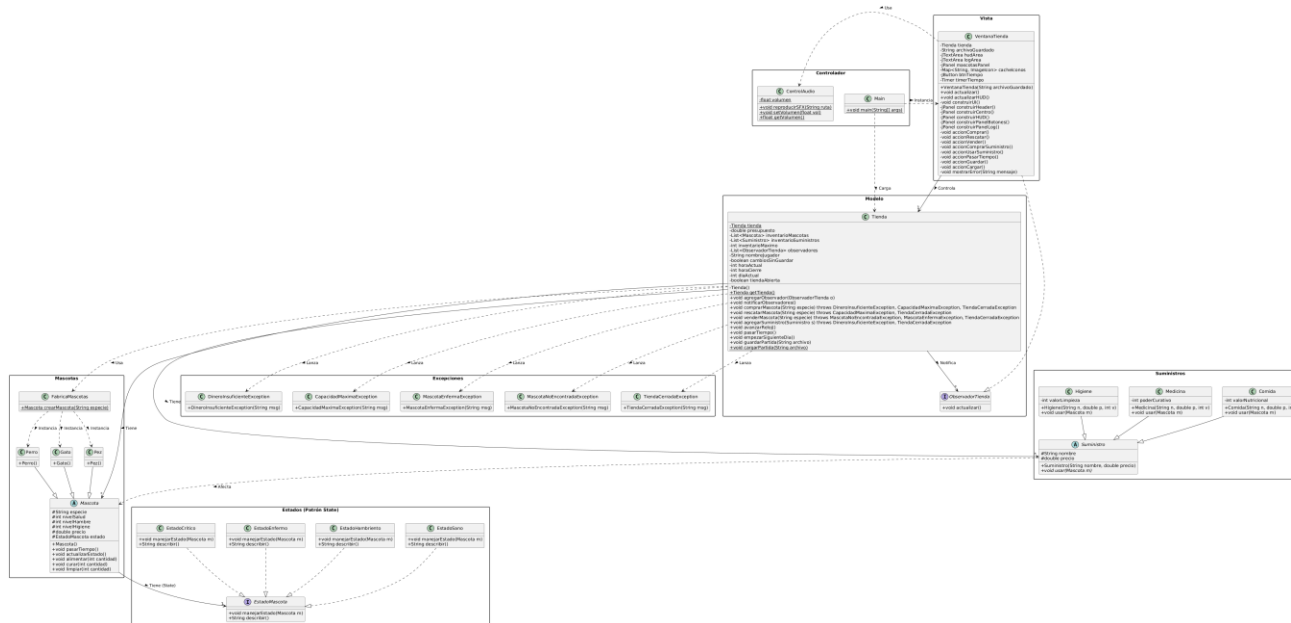
### ¿Por qué transient?

Tienda implements Serializable, si la lista de observadores no fuera transient, guardar la partida intentaría serializar VentanaTienda (un JFrame) y lanzaría NotSerializableException. Con transient, la lista se descarta al guardar y VentanaTienda se re-registra automáticamente al cargar (accionCargar()) llama a agregarObservador(this) sobre la nueva instancia).

## Semana 4 - Final

El objetivo principal fue implementar mecánicas de tiempo y penalizaciones, a la par de asegurar que la experiencia del usuario (UX) fuera a prueba de errores.

## Diagrama UML Semana 4 (LINKEADO DENTRO DE REPOSITORIO DE GITHUB)



## Personalización y UX Inicial

**Sistema de Perfil del Jugador:** Se implementó una ventana de diálogo nativa al iniciar una partida nueva (cuando no hay archivo .dat guardado) que le pregunta el nombre al jugador. Este nombre se asigna al Singleton Tienda, se refleja en el título de la ventana y se guarda persistentemente. Si el jugador lo deja en blanco, se asume por defecto "Jefe/a de la tienda".

**Prevención de Pérdida de Datos (Interceptación del Cierre):** Se modificó la operación por defecto de la ventana (DO\_NOTHING\_ON\_CLOSE) acoplando un WindowAdapter. Ahora, gracias a la bandera booleana cambiosSinGuardar integrada en el modelo, si el jugador intenta cerrar el juego en la "X" habiendo hecho modificaciones sin guardar, el sistema intercepta el cierre y lanza una alerta crítica (JOptionPane.WARNING\_MESSAGE) preguntando si está seguro de querer salir perdiendo su progreso. Al guardar la partida exitosamente, la bandera se reinicia.

|                    |                                                                                                                                                       |
|--------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------|
| Tienda.java        | Se agregaron las variables nombreJugador y cambiosSinGuardar (esta última con la etiqueta transient para que no afecte la serialización de guardado). |
| VentanaTienda.java | Se agregó un JOptionPane.showInputDialog en el constructor, y se implementó un WindowAdapter anónimo en construirUI()                                 |

|  |                                                         |
|--|---------------------------------------------------------|
|  | sobreescribiendo el método <code>windowClosing</code> . |
|--|---------------------------------------------------------|

## El Reloj In-Game y el Motor de Eventos (Swing Timer)

Se integró un sistema de tiempo dinámico usando `javax.swing.Timer`.

**Escala de Tiempo:** El temporizador se ejecuta en un hilo de fondo (background thread). Cada 10 segundos reales, dispara un evento que avanza el reloj de la tienda en 1 hora in-game.

**Jornada Laboral:** La tienda implementa el concepto de Días. Cada día inicia automáticamente a las 08:00 y cierra a las 20:00. Un día completo dura exactamente 2 minutos reales, obligando al jugador a tomar decisiones de gestión rápidas bajo presión.

**Penalizaciones Automatizadas:** Con cada tick del reloj (cada hora), el modelo invoca los métodos de deterioro natural: las mascotas reducen su salud e higiene, y aumentan su hambre.

|                    |                                                                                                                                                                                                                                                  |
|--------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| VentanaTienda.java | Se instanció y configuró un objeto de la clase <code>javax.swing.Timer</code> dentro del método <code>construirUI()</code>                                                                                                                       |
| Tienda.java        | Se añadieron las variables de estado <code>horaActual</code> , <code>horaCierre</code> , <code>diaActual</code> , y <code>tiendaAbierta</code> . Además, se crearon los métodos <code>avanzarReloj()</code> y <code>empezarSiguienteDia()</code> |

## Excepciones de Negocio y Bloqueo de Estados

La arquitectura de manejo de errores se expandió creando una nueva excepción chequeada personalizada:

**TiendaCerradaException:** Hereda de `Exception`.

**Validación de Capa de Negocio:** Cuando el reloj marca las 20:00, la variable booleana `tiendaAbierta` pasa a `false`. Todos los métodos que alteran la tienda (`comprarMascota`, `venderMascota`, `rescatarMascota`, `agregarSuministro`) ahora validan el estado de la tienda. Si está cerrada, arrojan la `TiendaCerradaException` interceptando la operación y obligando a la Vista a notificar al jugador mediante un cuadro de diálogo rojo que el horario comercial ha terminado.

|                             |                                                                                                                                                                                                                                   |
|-----------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| TiendaCerradaException.java | Clase que hereda de Exception para representar el error de horario comercial.                                                                                                                                                     |
| Tienda.java                 | Las firmas y la lógica interna de los métodos comprarMascota, venderMascota, rescatarMascota y agregarSuministro fueron actualizadas para incluir la validación if (!tiendaAbierta) y la directiva throws TiendaCerradaException. |
| VentanaTienda.java          | Se agregaron bloques catch (TiendaCerradaException ex) a todos los métodos de los botones (accionComprar, accionRescatar, etc.) para atrapar el error e informar al jugador mediante mostrarError().                              |

## UI Reactiva y Botón Contextual

El HUD fue expandido para mostrar la métrica temporal en vivo: DÍA: X | RELOJ: XX:00 | ESTADO: ABIERTA/CERRADA. Además, el manejo de eventos de UI se volvió State-Aware (consciente del estado de este mismo)

**Botón Inteligente:** El botón de la interfaz cambia de texto y función dinámicamente. Mientras el estado es ABIERTA, el botón es rojo y dice "Terminar Día".

**Mecánica de Salto de Tiempo:** Si el jugador hace clic en "Terminar Día", el sistema calcula cuántas horas faltan para las 20:00 y las adelanta todas de golpe usando un bucle. Esto aplica de forma brutal e instantánea todo el hambre y la pérdida de salud que las mascotas habrían sufrido en esas horas saltadas.

Al cerrar la tienda (ya sea natural o saltando el tiempo), el botón se transforma en "Empezar Siguiete Día", lo cual reinicia el reloj a las 08:00, avanza el contador de días y vuelve a poner tiendaAbierta = true.

|                                                                                                                                                                                                                                                                                                                                                                                                                                              |
|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| VentanaTienda.java                                                                                                                                                                                                                                                                                                                                                                                                                           |
| <p>El método actualizarHUD() fue refactorizado para redibujar la cabecera e incluir las variables de día y hora.</p> <p>También se modificó accionPasarTiempo() introduciendo un bucle for que salta todas las horas restantes de golpe (hasta llegar a 20) llamando múltiples veces a avanzarReloj(). Se implementó un condicional if para alterar el texto de btnTiempo basándose en el booleano devuelto por tienda.isTiendaAbierta()</p> |

## Decisiones de Diseño y Evolución de la Temática

Si bien cuando se fue desarrollando todo el proyecto, hubo muchas decisiones que tomar por no decir que cada paso que hacíamos era elegir 1 entre muchos caminos posibles, pero tomamos los que igual transformarían el proyecto de un simple inventario transaccional a un verdadero simulador de gestión de recursos y tiempo

Por lo que nos gustaría en especial destacar los siguientes

### Gestión de Crisis vs. Economía Básica (Mecánica de Rescate)

Inicialmente, la temática apuntaba a comprar animales y venderlos más caros. Para darle profundidad narrativa y estratégica, decidimos incorporar la mecánica de Rescate. Permitir al jugador adquirir mascotas gratis, pero que llegaran en un "Estado Crítico". Esto rompió el ciclo monótono de comprar/vender, obligando al jugador a sopesar el costo-beneficio de gastar en "Medicina" y usar tiempo para curar al animal antes de poder rentabilizarlo, aportando un dilema ético y estratégico a la gestión de la tienda.

### Mascotas "Vivas" mediante el Patrón State

Para que el simulador se sintiera real, los animales no podían ser simples variables de datos estáticas; necesitaban reaccionar a su entorno. Implementar el Patrón de Diseño State (EstadoSano, EstadoEnfermo, etc.) para encapsular el comportamiento de las mascotas. Las mascotas adquirieron "vida". Dependiendo de sus estadísticas de hambre y salud, su estado muta dinámicamente, impidiendo que sean vendidas si se encuentran enfermas. Esto transformó el juego en una experiencia de cuidado activo.

### Presión Psicológica mediante Tiempo Real (Reloj In-Game)

Una de las decisiones más arriesgadas e innovadoras de la Semana 4 fue abandonar el modelo tradicional por "turnos".

Integrar un Timer asíncrono que hace avanzar el reloj del juego (1 hora in-game cada 10/20 segundos reales) con un horario límite de cierre (20:00 hrs). Añadió una capa de "urgencia" típica de los juegos de gestión tipo Tycoon. El jugador ya no puede quedarse inactivo (AFK); si ignora a los animales, el tiempo pasará, las mascotas enfermarán y la tienda cerrará, forzando a priorizar acciones rápidamente.

### Mapeo del Dominio Comercial en el Código (Excepciones)

Decidimos que los errores del código debían reflejar los problemas de la vida real de una tienda de mascotas. En lugar de devolver booleanos (true/false) al fallar una compra o usar excepciones genéricas, creamos un ecosistema de excepciones de negocio

(DineroInsuficienteException, CapacidadMaximaException, TiendaCerradaException). El código fuente se volvió un reflejo exacto del negocio. Si el jugador intenta vender a las 21:00, el sistema no suelta un error técnico, sino que le notifica coherentemente: "La tienda está cerrada. Pasa al siguiente día".

## Estética de "Dashboard" Gerencial (HUD)

Implementar una paleta de colores oscura (Dark Mode), fuentes monoespaciadas y un panel central tipo HUD (Head-Up Display) con un registro de logs a lo consola clásica. Pone al jugador verdaderamente en el papel de "Jefe/a de la tienda" operando un sistema de gestión de inventario profesional y dinámico, mejorando drásticamente la inmersión del usuario final.

## Especialización de Consumibles (Polimorfismo Aplicado al Gameplay)

En lugar de tener un "ítem genérico" que subiera todas las estadísticas mágicamente, decidimos separar los suministros en categorías estrictas: Comida, Medicina e Higiene.

Hacer que la clase abstracta Suministro obligue a implementar el método usar(Mascota m), pero que cada subclase afecte una estadística distinta, esto añade un componente de "diagnóstico" al rol del jugador. No basta con comprar suministros al azar; el jugador debe observar qué necesidad específica tiene cada animal (hambre, enfermedad o suciedad) y aplicar el remedio correcto, haciendo el juego más realista y estratégico.

## Límite de Aforo (Gestión de Espacio Físico)

Podríamos haber usado una lista infinita para almacenar a las mascotas, pero optamos por imponer una restricción artificial de espacio (inventarioMaximo = 5). Lanzar una CapacidadMaximaException si se intenta comprar o rescatar un animal sin espacio físico en la tienda. Simula los costos de mantenimiento y las restricciones de espacio físico de una tienda real. Previene que el jugador acumule animales infinitamente para venderlos todos de golpe, forzando una constante rotación de inventario y toma de decisiones sobre qué animal vender para "hacer espacio" a un nuevo rescate.

## Permanencia del Negocio (Serialización y Singleton)

La tienda no podía reiniciarse cada vez que se cerraba el programa, el progreso del negocio debía ser persistente.

Utilizar el patrón Singleton para asegurar que exista una única y misma franquicia ejecutándose, combinada con la Serialización de objetos nativa de Java (.dat), le otorga "peso" a las decisiones del jugador. Los errores financieros o el abandono de mascotas

quedan grabados. Además, la necesidad de hacer clic en "Guardar Partida" simula la responsabilidad gerencial de "hacer el corte de caja" al final del día.

## Feedback Sensorial y Recompensas (Integración de Audio)

La gestión puramente visual o matemática puede volverse tediosa; los simuladores comerciales dependen de la dopamina visual y sonora. Desarrollar la clase utilitaria `ControlAudio` ejecutada en hilos paralelos para no bloquear el reloj, y vincular sonidos específicos (como el de una caja registradora) a los métodos de compra y venta. El efecto de sonido de la caja registradora al vender con éxito recompensa auditivamente al jugador, haciendo que las transacciones económicas se sientan tangibles y satisfactorias.

## Auditoría en Tiempo Real (Consola de Logs)

Mostrar un Pop-up (ventana emergente) por cada animal que siente hambre habría arruinado el flujo del juego, especialmente con el paso del tiempo automatizado. Implementar un `JTextArea` inferior que actúa como una bitácora inmutable (`logArea`) donde se imprimen todos los eventos sin detener el reloj, este funciona como un sistema de "Registro de Eventos" o auditoría del sistema (muy típico en software empresarial real). Le permite al "Jefe/a de la tienda" leer el historial reciente de lo que ocurrió en su ausencia y llevar un control estricto de las operaciones financieras del día sin frustraciones en la interfaz.

## Problemas Identificados y Autocrítica de Gestión

Durante el ciclo de vida del desarrollo de este simulador, nos enfrentamos a múltiples desafíos tanto de nivel de desarrollo, arquitectónicos u gestión de los flujos de trabajo, irlos identificando fue clave para ayudar a solventarlos.

### Problemas Identificados

#### Acoplamiento en la Transición a GUI

Durante la Semana 3, migrar de una interfaz por consola a una interfaz gráfica (`Swing`) reveló que ciertas lógicas de negocio estaban fuertemente acopladas a la presentación. Separar el proyecto en un patrón MVC estricto tomó más tiempo de refactorización del estimado.

#### Problemas de Persistencia y Serialización

Al implementar el guardado de partidas, nos encontramos con excepciones críticas (`NotSerializableException`). El error ocurrió porque el patrón `Observer` vinculaba la Vista

(VentanaTienda) al Modelo (Tienda), provocando que Java intentara guardar la ventana gráfica en el archivo binario. Esto se solucionó investigando e implementando la palabra clave transient.

## Desincronización del IDE y Falsos Positivos

Al implementar las excepciones de tiempo (TiendaCerradaException), enfrentamos problemas donde los entornos de desarrollo (IDE) no refrescaban los cambios en disco a tiempo. Esto nos llevó a aplicar "Quick-Fixes" erróneos que rompieron las funciones Lambda de los botones, forzándonos a compilar manualmente y limpiar la caché.

## Conflictos de Control de Versiones (Git)

El trabajo colaborativo generó "Merge Conflicts" severos (MERGE\_HEAD exists), bloqueando los repositorios locales al intentar subir (push) código modificado simultáneamente en los mismos archivos.

## Autocrítica y Gestión del Proyecto

### Falta de una Estrategia de Ramas (Git Flow)

Gran parte de nuestros problemas de integración ocurrieron por trabajar y hacer commits directamente sobre la misma rama principal. Debimos haber adoptado una estrategia de ramas por característica (feature branches) para aislar el trabajo en progreso y hacer Pull Requests limpios.

### Planificación Arquitectónica Temprana

Si bien el código final es robusto, debimos haber planificado la arquitectura orientada a eventos (MVC + Observer) desde la Semana 1. Construir primero para consola y luego adaptar a GUI generó un retrabajo que se pudo haber evitado con una mejor visión a largo plazo.

### Pruebas de Integración Continuas

Subestimamos el tiempo que tomaría integrar mecánicas dinámicas (como el Timer en tiempo real) con la lógica estática. De cara a futuros proyectos, es imperativo establecer ciclos de prueba (Testing) más rigurosos antes de dar por terminada una fase.